

Section 1: Fairness requirements

Two fairness harms were identified in our recommender system. The first is popularity bias. The popularity model recommends the same 20 movies to every user regardless of their preferences. This means the vast majority of the 3706 movies in the catalog get zero exposure.

The second is activity bias. Both models perform significantly worse for users with few ratings. In our Milestone 3 subpopulation analysis, KNN achieved an HR@20 of 0.367 for light users compared to 0.791 for heavy users.

We defined one system-level and one model-level fairness requirement with measurable metrics.

System-level requirement (catalog coverage): at least 50% of all movies in the catalog should appear in at least one user's top-20 recommendations. This ensures the system does not ignore the long tail of content.

Model-level requirement (equity gap ratio): HR@20 for the bottom tercile of users by rating count should be at least 50% of the top tercile's HR@20. This ensures the model does not disproportionately underserve inactive users.

Section 2: Fairness improvements

For popularity bias we propose a diversity re-ranking step. After generating the initial top-K recommendations, replace a configurable fraction (e.g. 20%) with randomly sampled movies from outside the top-100 most popular. KNN only covers 150 of 3706 movies (4.0%) and popularity covers just 20 (0.5%), so even modest diversification would significantly improve coverage.

For activity bias we propose a hybrid serving strategy. Users with fewer than 20 ratings receive popularity-based recommendations since KNN has insufficient signal for them. Users above that threshold receive KNN results. As users accumulate more ratings they gradually transition to fully personalized recommendations.

For ongoing monitoring we would track catalog coverage and per-tercile HR@20 after each retraining cycle. If either metric degrades below its threshold an alert would notify the team to investigate.

Section 3: Fairness analysis

We evaluated both fairness requirements using data from our trained models and Milestone 3 evaluation results.

For catalog coverage, we generated top-20 recommendations for 1000 sampled users and counted unique movies across all lists. KNN recommended 150 unique movies out of 3706 total

(4.0% coverage). Popularity recommended 20 unique movies (0.5% coverage). Both fail the 50% threshold by a wide margin. The system heavily concentrates recommendations on a small subset of the catalog.

For the equity gap, subpopulation analysis from Milestone 3 showed the following HR@20 by user activity tercile. KNN: light 0.367, medium 0.641, heavy 0.791. Equity gap ratio = 0.464, which fails the 0.50 threshold. Popularity: light 0.285, medium 0.480, heavy 0.549. Equity gap ratio = 0.519, which barely passes.

```

=== SUBPOPULATION ANALYSIS (KNN) ===
Users split into terciles by training set rating count.

```

Group	Users	HR@20	NDCG@20
Light	330	0.3667	0.0626
Medium	331	0.6405	0.0933
Heavy	339	0.7906	0.1955

```

=== SUBPOPULATION ANALYSIS (Popularity) ===

```

Group	Users	HR@20	NDCG@20
Light	330	0.2848	0.0401
Medium	331	0.4804	0.0485
Heavy	339	0.5487	0.0599

```

=== CATALOG COVERAGE ===
Total movies in catalog:      3706
KNN unique movies recommended: 150
KNN catalog coverage:         4.0%
Popularity unique movies:     20
Popularity catalog coverage:  0.5%

=== EQUITY GAP ===
(from M3 subpopulation analysis)
KNN HR@20 light users:      0.3667
KNN HR@20 heavy users:     0.7906
Equity gap ratio:           0.4638
Threshold (>=0.50):        FAIL
Pop HR@20 light users:     0.2848
Pop HR@20 heavy users:     0.5487
Equity gap ratio:           0.5190
Threshold (>=0.50):        PASS

```

KNN provides better overall accuracy but has a wider equity gap and extremely low catalog coverage. The popularity model has a slightly better equity ratio but even worse coverage. Neither model meets our system-level fairness requirement. Addressing these gaps would

require the diversity re-ranking and hybrid serving strategies described above.

Scripts:

https://github.com/TheFelixGomez/cot6930-final-project/blob/main/scripts/fairness_analysis.py

Catalog coverage: standard diversity metric measuring the fraction of available items appearing in recommendations (Herlocker et al., "Evaluating Collaborative Filtering Recommender Systems," ACM TOIS 22(1), 2004).

Equity gap ratio: adapted from disparate impact analysis measuring performance ratio between disadvantaged and advantaged groups.

Subpopulation evaluation methodology: chronological train/test split with tercile-based user grouping (see Milestone 3 offline evaluation).

Section 4: Feedback loops

Tail Starvation

Upon analysis, the model has difficulty recommending niche movies due to the heavy bias of recommending top-rated movies. Because it restricts itself to the movies that it knows well, this leads to near-zero interaction with niche/unpopular movies within the kafka stream. Ideas for detection include monitoring the coverage rate among the movie catalog so as to determine which movies are being covered and prioritize it by a threshold to expand the model's capabilities.

Popularity Echo

The popularity model tends to recommend items solely based on interaction counts. This means that users that select the more popular movies, those movies will stay at the top of the queue for recommendation. Ideas for detection include using the Gini Coefficient to measure the inequality among popular and niche movie titles, determining the quality of the recommendations, and further confirming whether the recommendations continue to be prioritized from the model.

Section 5: Loop analysis

```
== FEEDBACK LOOP ANALYSIS ==
Metric                        Value
-----
Total Recommendations         2750
Unique Movies Seen           14
Catalog Coverage (%)         0.38      %
Gini Coefficient              0.2314

Decision: Tail Starvation detected (Low exploration).
```

Findings

- The model recommends only 14 movies out of the 2750 recommendation queries and the low coverage rate for the movie catalog with 38%. This indicates that the majority of items are less likely to appear in future training data.
- The Gini Coefficient of .23 shows us that distribution of the 14 unique movies were distributed evenly, but still suffers from the lack of concentration of movies recommended from the dataset.
- The 14 movies that are served created a closed loop of feedback. This means that the gcl.rate and gcl.watch topics will contain feedback for those items only, preventing learning from the rest of the movie catalog.

Section 6: Security

API Layer (FastAPI)

Threat

Unauthorized access to telemetry and Denial of Service (DoS).

Mitigation

We implemented HTTP Basic Auth for the /metrics endpoint using secrets.compare_digest to prevent timing attacks.

To mitigate Denial of Service (DoS) and automated data poisoning attacks, we implemented application-layer rate limiting. We integrated the package fastapi-limiter to track client IP addresses across all active containers. All endpoints are strictly capped at 1k requests per minute per client. Any traffic exceeding this triggers an HTTP 429 Too Many Requests

response, successfully neutralizing botnets attempting to exhaust our compute resources.

Message Broker Layer (Apache Kafka)

Threat

Interception of telemetry data (Man-in-the-Middle) and injection of malformed data.

Mitigation

We implemented strict SSL/TLS encryption for all Kafka connections (using kafka-ca.pem). Additionally, stream_ingestor.py enforces Schema Checks using jsonschema against schemas/event.json. Any invalid payloads are quarantined into a Dead Letter Queue (gcl.reco_requests.dlq), preventing bad data from reaching the storage layer.

Model Registry (Cloudflare R2)

Threat

Unauthorized modification or deletion of model artifacts (e.g., replacing knn.pkl with a malicious file).

Mitigation

Access is strictly controlled via IAM credentials (R2_ACCESS_KEY_ID and R2_SECRET_KEY). The stream_ingestor.py only writes, and main.py only reads, adhering to the principle of least privilege.

Model-Specific Attack: Rating Spam / Data Poisoning

Threat

A malicious actor or botnet floods the system with fake recommendation requests or telemetry events aimed at artificially inflating the "popularity" model for a specific set of movies, or skewing the KNN neighborhood vectors.

Mitigation

1. Schema Validation & Approvals: Only correctly formatted events with valid probe_ids pass the ingestor.
2. Anomaly Detection: Monitoring request rates to detect unnatural spikes in activity that indicate automated spamming.

Section 7: Security analysis

To analyze our system's resilience against Denial of Service (DoS) and data poisoning attacks, we simulated a high-volume botnet attack against the API gateway. Using Apache Bench, we flooded the server with 1,500 requests at a high concurrency level of 150 simultaneous connections over a 10 seconds window.

```
Server Software:      uvicorn
Server Hostname:      cot6930-final-project.onrender.com
Server Port:          443
SSL/TLS Protocol:     TLSv1.2,ECDHE-ECDSA-AES128-GCM-SHA256,256,128
TLS Server Name:      cot6930-final-project.onrender.com

Document Path:        /
Document Length:      26 bytes

Concurrency Level:     150
Time taken for tests:  13.668 seconds
Complete requests:     1500
Failed requests:        500
  (Connect: 0, Receive: 0, Length: 500, Exceptions: 0)
Non-2xx responses:     500
Total transferred:     495500 bytes
HTML transferred:      41000 bytes
Requests per second:   109.75 [#/sec] (mean)
Time per request:      1366.795 [ms] (mean)
Time per request:      9.112 [ms] (mean, across all concurrent requests)
Transfer rate:         35.40 [Kbytes/sec] received
```

As evidenced in the attached terminal capture, our rate limiter successfully detected and neutralized the anomaly. Out of the 1,500 requests, exactly 500 were intercepted and dropped, resulting in "Non-2xx responses" (specifically, HTTP 429 Too Many Requests). Only the initial handful of requests permitted by our strict threshold of 1000 per minute succeeded.

To further verify the efficacy of our mitigations, we analyzed the raw Prometheus telemetry exposed at our /metrics endpoint during the attack window. The telemetry successfully captured the anomaly. As evidenced by our instrumentation, the `http_requests_total` metric recorded a massive spike in blocked traffic, logging `http_requests_total{handler="/",method="GET",status="4xx"} 1368.0`. This internal metric proves that our rate limiter correctly intercepted and dropped the malicious requests at the application layer (via HTTP 429), neutralizing the attack before it could impact backend compute availability or poison the data lake.

```
http_requests_total{handler="none",method="HEAD",status="4xx"} 1.0
http_requests_total{handler="/",method="GET",status="2xx"} 3002.0
http_requests_total{handler="/metrics",method="GET",status="2xx"} 95.0
http_requests_total{handler="/ping",method="HEAD",status="2xx"} 46.0
http_requests_total{handler="/metrics",method="GET",status="4xx"} 2.0
http_requests_total{handler="none",method="GET",status="4xx"} 4.0
http_requests_total{handler="/docs",method="GET",status="2xx"} 1.0
http_requests_total{handler="/openapi.json",method="GET",status="2xx"} 1.0
http_requests_total{handler="/",method="GET",status="4xx"} 1368.0
http_requests_total{handler="/recommend",method="POST",status="2xx"} 3.0
```

This telemetry proves that our security mitigation is effective. Any automated attempts to overwhelm the compute resources or artificially poison the recommender data lake are immediately throttled at the application layer, ensuring system availability and data integrity.

Section 8: Final demo

Video

<https://youtu.be/wjCiogxReM8>

Presentation

https://docs.google.com/presentation/d/1UgzN8oU7yhKpFmaS_VFTpx45SZh3ZPndYFVia2FUViY/edit

Section 9: Reflection

Hardest Parts

Embedding the Kafka consumer inside the FastAPI lifecycle (main.py) was difficult due to FastAPI being able to run multiple work processes, meaning that the team had to avoid running which means multiple consumer threads could unintentionally spin up and cause offset conflicts or duplicate processing if not carefully managed.

Getting the evaluation pipeline performing was a challenge. The initial KNN implementation ran for over 20 minutes because it did per item neighbor lookups for every user. Switching to a matrix multiplication approach cut that to under two minutes, reinforcing that algorithm choice matters more than hardware at scale.

Managing the model artifact handoff between local development and the deployed container was another pain point. The trained.pkl files were in .gitignore so they never made it into the Docker image. The API returned errors in production while working perfectly locally. This was a

concrete lesson in why "works on my machine" is a real production problem.

Configuring the CI/CD environment was also a challenge as a failure on the coverage checks would prevent future iterations from being implemented, especially if the tests were written incorrectly. Making sure that the tests were correct and covered essential variables (Unit Tests, Drift Detection, Backpressure, etc.) was crucial over the course of development.

Fragilities

The API contains an in-memory rate limiter, which tends to be fragile in cloud environments such as Render. This means that if the platform scales horizontally, the in-memory state will not be shared across new containers, being susceptible to privilege escalation.

The model serving pipeline is also fragile. If the R2 bucket is unreachable on startup, the container falls back to locally bundled models which may be outdated. There is no automatic retry or health check that verifies the loaded model version matches what the registry says is latest.

Probes periodically hit the API to determine health as well as reveal personalized metrics, and often the render will fail to recognize these probes and time out, affecting the CI/CD pipeline. Making sure the render cloud environment variables as well as scripts is updated to ensure that probes and other CI/CD workflows are working as intended.

Productionization Plan

To make this product production-ready, the team suggested implementing a redis-backed distribution instead of a rate-limiter to ensure IP client tracking is persistent and synchronized across all platforms. We also recommend allocating the rate-limiting and authentication to AWS API Gateway or any dedicated external API gateway.

For the model pipeline, we would add a startup health check that verifies the loaded model version against the registry and blocks traffic until a valid model is confirmed. We would also add automated offline evaluation as a gate in the retraining pipeline

The CI/CD pipeline can integrate event-driven automation to trigger coverage as well as drift detection tests when they meet a specific threshold (70% or better), to which model retraining would activate and update the model version accordingly.

"if we redid it" notes

If we redid this project, we would set up the model artifact pipeline on day one, train locally, upload to R2, have the container pull on startup. We eventually got there but it blocked other team members in the middle milestones and affected results. We would also include NDCG as an evaluation metric from the start since it gives a much better picture of ranking quality than hit rate alone.

Reflection from team

Tight coordination was required to ensure that applying strict rate limits and dropping malformed telemetry payloads didn't accidentally block the valid events needed for the A/B testing and model hot-swapping.

Splitting work by role, which was infrastructure, models, and data pipeline, worked well because each person could operate independently most of the time. The main friction with machine learning was when one person's deliverable blocked another, especially around deployment.